

found on the system by the Linux device driver subsystem.

To understand how to write I2C device information and the I2C driver, let's consider an example of a system in which there are two devices connected on the I2C bus. A description of these devices is given below.

Device 1

Device type: EEPROM
 Device name: eeprom_xyz
 Device I2C address: 0x30
 Device interrupt number: 4
 Device bus number: 2

Device 2

Device type: Analogue to digital converter
 Device name: adc_xyz
 Device I2C address: 0x31
 Device interrupt number: Not available
 Device bus number: 2

Writing the I2C device file

I2C devices connected on the system are represented by *struct i2c_client*. This structure is not directly defined but, instead, *struct i2c_board_info* is defined in the board file. *struct i2c_client* is defined using *struct i2c_board_info* structure by the Linux I2C subsystem, the fields of the *i2c_board_info* object is copied to *i2c_client* object created.



Note: Board files reside in *arch/* folder in Linux. For example, the board file for the ATSTK1000 board of the AVR32 architecture is *arch/avr32/boards/atstk1000.c* and the board file for Beagle Board of ARM OMAP3 architecture is *arch/arm/mach-omap2/board-omap3beagle.c*.

struct i2c_board_info (defined in *include/linux/i2c.h*) has the following important fields.

type: This is the name of the I2C device for which this structure is defined. This will be copied to the name field of *i2c_client* object created by the I2C subsystem.

addr: This is the address of the I2C device. This field will be copied to address the field of *i2c_client* object created by the I2C subsystem.

irq: This is the interrupt number of the I2C device. This field will be copied to the *irq* field of the *i2c_client* object created by the I2C subsystem.

An array of *struct i2c_board_info* object is created, where each object represents the I2C device connected on the bus. For our example system, the *i2c_board_info* object is written as follows:

```
static struct i2c_board_info xyz_devices[] = {
{
.type = "eeprom_xyz",
.addr = 0x30,
```

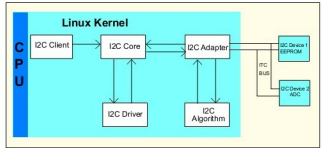


Figure 1: Linux I2C system

```
.irq = 4,
},
{
.type = "adc_xyz",
.addr = 0x31,
},
};
```

I2C device registration

I2C device registration is a process with which the kernel is informed about the device present on the I2C bus. The I2C device is registered using the *struct i2c_board_info* object defined. The kernel gets information about the device's address, bus number and name of the device being registered. Once the kernel gets this information, it stores this information in its global linked list *__i2c_board_list*, and when the *i2c_adapter* which represents this bus is registered, the kernel creates the *i2c_client* object from this *i2c_board_info* object.

I2C device registration is done in the board *init* code present in the board file. I2C devices are registered in the Linux kernel using the following two methods.

Case 1: In most cases, the bus number on which the device is connected is known; in this case the device is registered using the bus number. When the bus number is known, I2C devices are registered using the following API:

```
int i2c_register_board_info(int busnum, struct i2c_board_info
*info, unsigned len);
```

...where,
busnum = the number of the bus on which the device is connected. This will be used to identify the *i2c_adapter* object for the device.

info = array of *struct i2c_board_info* object, which consists of information of all the devices present in the bus.
len = number of elements in the info array.

For our example system, I2C devices are registered as follows:

```
i2c_register_board_info(2, xyz_devices, ARRAY_SIZE(xyz_
devices));
```